Motivation Modern large language models (LLMs) demonstrate remarkable capabilities in reasoning tasks. But, training these reasoning-capable models at scale demands substantial computational resources. Reinforcement learning (RL) presents a promising avenue to enhance reasoning by rewarding intermediate "thinking" steps. Unfortunately, conventional RL training is often centralized and constrained by communication overhead. This project introduces an approach leveraging asynchronous RL on globally distributed inference nodes within a decentralized framework to train reasoning models efficiently. A key insight is that for tasks involving long "thinking chains," test-time compute (inference) substantially outweighs training compute, making RL training particularly amenable to decentralization.

Method We propose a decentralized two-step asynchronous RL framework. Training is based on Group-Relative Policy Optimization (GRPO) **?** with binary task rewards for mathematics and coding problems, supplemented by length rewards to encourage adherence to a specified "thinking budget." In our decentralized two-step asynchronous setup, trainers push updated model weights to a distribution system (Shardcast), and inference nodes pull weights from two steps prior to generate rollouts, effectively mitigating issues of staleness while maximizing parallelism. Methodological contributions include offline filtering of training data to remove problems that are too easy or too hard for the base model, and online filtering to make sure training batches contain non-zero advantages, thereby improving the learning signal. We also introduce two-sided GRPO clipping by adding an upper bound δ on the probability ratio when advantages are negative, enhancing training stability. Further stability is achieved through aggressive gradient clipping and by addressing issues related to `torch.compile`.

Implementation We conducted experiments using the QwQ-32B model (32 billion parameters) with a context length of up to 32K tokens. Two main experiments were performed: a "Short Experiment" with target thinking lengths of {1000, 2000, 3000, 4000} tokens, and a "Long Experiment" with target lengths of {2000, 4000, 6000, 8000, 10000} tokens. Training utilized GRPO with specific hyperparameters ($\varepsilon = 0.2, \delta = 4,$ KL coef $= 0.001, \alpha = 0.0003,$ entropy coef $=$ 1e-4), a learning rate of 3e-7, and sequence packing for efficiency. Two-step asynchrony was maintained to overlap inference and training.

Results Ablation studies on asynchrony (1 to 4-step delay) showed negligible performance loss compared to synchronous training. In the main experiments with QwQ-32B (termed INTELLECT-2 runs), we observed a substantial rise in task rewards for both Short and Long Experiment configurations. Length penalties showed a slower decline but trended downwards. The model, trained with these methods, demonstrated improved performance over the QwQ-32B baseline on AIME (AIME24: 78.8 vs 76.6) and LiveCode (v5) (67.8 vs 66.1) benchmarks.

Discussion The results underscore the potential of decentralized RL for scaling reasoning models. As reasoning length increases, inference FLOPs dominate training FLOPs, making decentralization increasingly efficient due to the embarrassingly parallel nature of inference. This approach allows for harnessing globally distributed, heterogeneous compute resources.

Conclusion This work successfully demonstrates the feasibility and benefits of training large reasoning models using asynchronous RL in a decentralized setting. The proposed methods for data filtering, GRPO modification, and stability enhancements contribute to robust and efficient training. Future work will explore integrating tool-augmented reasoning and further scaling decentralized RL efforts.

Training Large Reasoning Models with Asynchronous Policies

Kushal Thaman
Department of Computer Science
Stanford University
kushalt@stanford.edu
Work done in collaboration with Justus Mattern & others at Prime Intellect

Abstract

Modern large language models (LLMs) excel at reasoning tasks, but training these models at scale is computationally intensive. Reinforcement learning (RL) can improve reasoning by rewarding intermediate "thinking" steps, yet traditional RL training is centralized and communication-bound. This paper introduces an asynchronous RL approach on globally distributed inference nodes within a decentralized framework for efficient training of reasoning models. We demonstrate that for long reasoning chains, inference compute dominates training compute, making RL amenable to decentralization. Our method, based on Group-Relative Policy Optimization (GRPO) with binary task rewards and length rewards, incorporates offline and online data filtering, a two-sided GRPO clipping mechanism, and other stability fixes. Experiments on a 32-billion parameter model (QwQ-32B), resulting in the INTELLECT-2 model, show substantial task reward improvements and superior performance on reasoning benchmarks compared to its baseline. We also show that asynchronous training with up to a 4-step delay incurs negligible performance loss.

1 Introduction

Large Language Models (LLMs) have shown impressive capabilities in complex reasoning tasks **??**. But, training models that are proficient in reasoning, especially at large scales, requires massive computational resources. Reinforcement Learning (RL) offers a powerful paradigm to enhance reasoning abilities by providing rewards for intermediate steps of "thinking" or problem-solving **??**. This is particularly effective for tasks like multi-step mathematical reasoning or code generation.

A substantial challenge with traditional RL training for LLMs is its centralized nature. Such setups often rely on large, co-located clusters of GPUs with high-speed interconnects, making them expensive and resource-intensive. Moreover, the synchronous nature of many RL algorithms (e.g., PPO **?**) can lead to communication bottlenecks, where training progress is gated by the slowest workers or by data transfer speeds.

We propose a shift towards asynchronous RL operating on globally distributed inference nodes within a decentralized framework. This approach is motivated by a key observation: for reasoning tasks that involve generating long chains of thought (e.g., thousands of tokens), the computational cost of inference (generating these rollouts) far exceeds the cost of the training updates themselves. As inference is an embarrassingly parallel task, it can be effectively distributed across many heterogeneous, geographically dispersed nodes.

Our work introduces a system that decouples inference generation from policy training. Fig. 1 illustrates the conceptual difference between synchronous RL, centralized asynchronous RL, and our proposed decentralized two-step asynchronous RL. In the latter, inference nodes may use slightly

stale policy weights (e.g., from two steps prior) to generate experience, while the central trainer continuously updates the policy with incoming data. This asynchrony helps to mask communication latencies and improve overall hardware utilization.

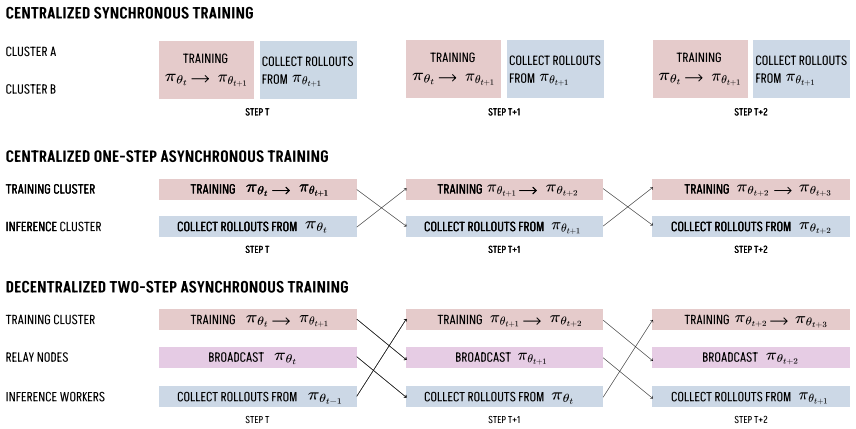


Figure 1: Comparison of Synchronous vs. Centralized Async vs. Decentralized Two-Step Async RL. In decentralized async RL, inference nodes (globally distributed) generate rollouts using slightly older policies, while trainers update the main policy.

This paper details our method, including data filtering techniques, modifications to the Group-Relative Policy Optimization (GRPO) algorithm for enhanced stability, and experimental results from training a 32-billion parameter model, QwQ-32B, which we refer to as INTELLECT-2.

2 Related Work

Improving the reasoning capabilities of LLMs through RL has been an active area of research. Models like AlphaCode and Minerva demonstrated the power of generating multiple solutions and selecting the best one. RL-based fine-tuning, often using PPO or its variants, has been employed to directly optimize models for task-specific rewards.

GRPO **?**, an extension of PPO, is designed for tasks where relative ranking within a group of generated samples is more informative than absolute scores. It has shown success in mathematical reasoning. Our work builds upon GRPO.

Decentralized and asynchronous training for deep learning is not new, with historic systems like DistBelief. More recently, asynchronous methods have been explored for RL and LLM training **??**. The focus on leveraging distributed inference for RL with long "thinking chains" is a key aspect of our approach, similar in spirit to systems that offload parts of the computation **?**.

The concept of a "thinking budget" and length rewards is explored in works like L1 Aggarwal et al. (2025), aiming to control the verbosity and computational cost of LLM responses during inference.

3 Method

Our approach focuses on training large reasoning models using asynchronous RL in a decentralized setting. This section details the core components of our method.

3.1 Asynchronous RL with GRPO

We base our training on Group-Relative Policy Optimization (GRPO) **?**. GRPO is suitable for tasks like mathematics and coding where verifying a solution yields a binary reward (correct/incorrect). We supplement these task rewards with length rewards to guide the model towards a specified "thinking budget" provided in the prompt (e.g., "Think for N tokens").

In our decentralized 2-Step asynchronous setup:

1. Trainers (centralized) compute policy updates and push new model weights to a distribution system (Shardcast).
2. Inference nodes (globally distributed, potentially heterogeneous) pull model weights. To mitigate staleness and make sure continuous operation, they pull weights from two steps prior to the current training step.
3. These inference nodes generate rollouts (reasoning traces and solutions).
4. Rollouts are sent back to the trainers for policy updates.

This two-step delay allows for substantial overlap between weight distribution, inference, and training, making the system robust to network latencies inherent in a global distribution.

Ablation on Asynchrony To validate the impact of policy staleness, we conducted ablation studies comparing synchronous RL with 1, 2, 3, and 4-step asynchronous RL on the Deepscaler math dataset **?**. As shown in Fig. 2, task reward trajectories indicate that up to a 4–step delay results in negligible performance loss compared to synchronous training. This finding is crucial as it supports the viability of highly asynchronous decentralized training.

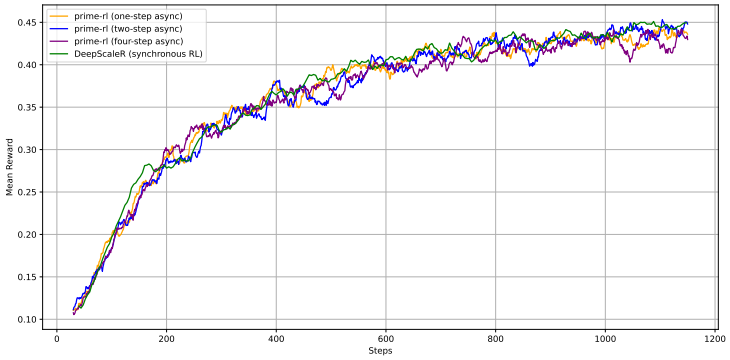
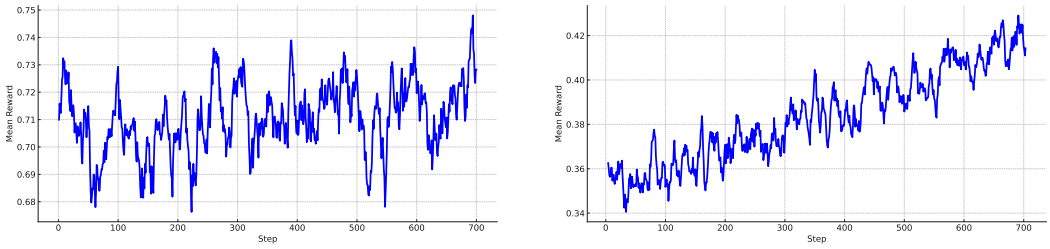


Figure 2: Task reward trajectory for synchronous vs. 1–4 step async on Deepscaler math dataset. Performance remains comparable even with a 4-step delay.

3.2 Data Filtering Strategies

Offline Filtering To accelerate learning and focus the model on appropriately challenging problems, we apply offline filtering to the math problem dataset. Based on the performance of a base model (e.g., pass@8 rate), we remove problems that are either too easy (pass@8 > 50%) or too difficult (pass@8 < 12.5%). Fig. 3 demonstrates that this filtering leads to faster reward improvement.



(a) Without Offline Filtering (b) With Offline Filtering

Figure 3: Impact of offline difficulty filtering on task reward for a 7B model on the Deepscaler math dataset. Filtering (b) leads to faster improvement compared to no filtering (a).

Online Filtering During training, we employ online filtering to make sure that each training batch provides a strong learning signal. We continue sampling rollouts until each problem in the batch has at least one successful and one unsuccessful attempt (or a variation leading to non-zero advantages). This naturally increases inference demand, which fits well with our decentralized inference setup.

3.3 Two-Sided GRPO Clipping

Standard PPO and GRPO objectives clip the probability ratio $r_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$ to prevent excessively large policy updates. The clipping is typically one-sided for negative advantages ($\hat{A} < 0$), meaning r_t can still become very large if it increases the objective (i.e., makes $r_t\hat{A}_t$ less negative). This can lead to instability.

We introduce an upper bound δ on the probability ratio r_t specifically when the advantage estimate $\hat{A}_t < 0$. This prevents unbounded updates and improves training stability. The standard PPO/GRPO objective component is $\text{clip}(r_t, 1-\varepsilon, 1+\varepsilon)\hat{A}_t$. Our modification effectively make sures that r_t does not exceed δ when $\hat{A}_t < 0$, by adjusting the term that would otherwise be $r_t\hat{A}_t$. The key terms involved are illustrated by the poster as:

$$\min\left(\frac{\pi_\theta}{\pi_{\text{old}}},\, \delta\right)\hat{A}_t \quad \text{(our modified term for negative } \hat{A}_t \text{ scenarios)}$$

versus the standard clipping term:

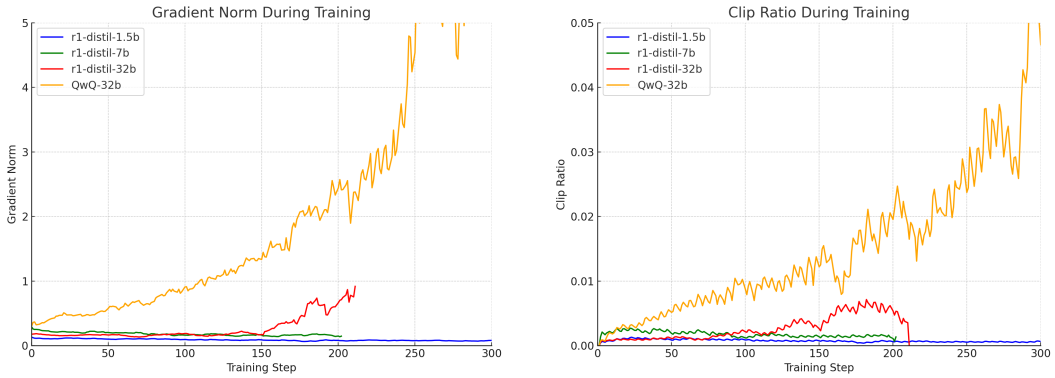
$$\text{clip}\left(\frac{\pi_\theta}{\pi_{\text{old}}},\, 1-\varepsilon,\, 1+\varepsilon\right)\hat{A}_t$$

This two-sided clipping, particularly the cap δ for negative advantages, helps stabilize training, especially for larger models.

3.4 Additional Stability Fixes

Training large models with RL can be prone to instabilities. We implemented several additional fixes:

- Aggressive Gradient Clipping: We clip gradients at a small value (e.g., 0.1) to curb the escalation of gradient norms, a phenomenon often observed in large model training that can presage collapse (see Fig. 4a). Fig. 4b shows the corresponding token clip ratio behavior.
- Disable `torch.compile`: We found that using `torch.compile` led to kernel instability and training collapse in some instances. Disabling it restored stability, albeit with a slight performance cost (see Fig. 6).
- Entropy Regularization: An entropy bonus is added to the loss to encourage exploration. We observed that after an initial drop, entropy can resurge, potentially leading to collapse if unchecked (Fig. 5). Careful tuning of the entropy coefficient is necessary.



(a) Gradient Norms vs. Training Steps

(b) Token Clip Ratio vs. Training Steps

Figure 4: Scaling Instabilities: Larger models exhibit rising gradients and clipping ratios that can precede training collapse. Plots show illustrative data.

4 Experimental Setup

We conducted experiments to evaluate our asynchronous RL training method.

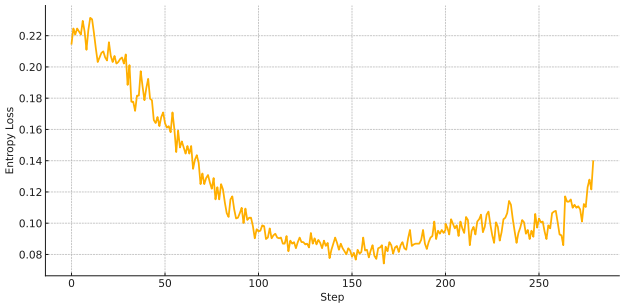
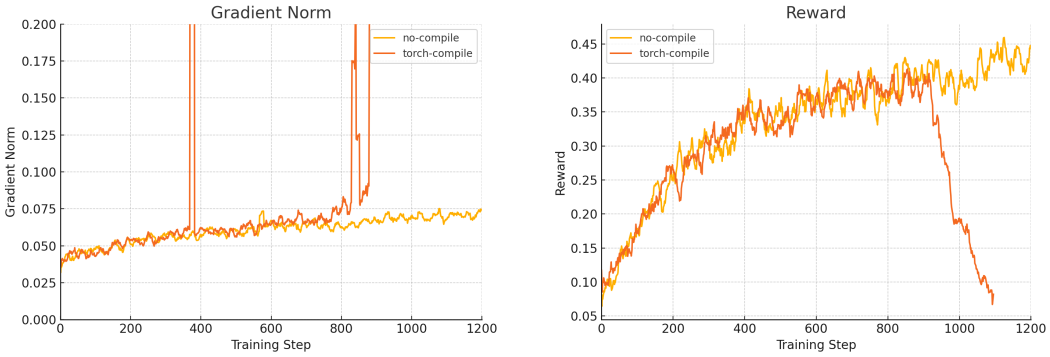


Figure 5: Entropy loss trajectory. After an initial drop, entropy can resurge, potentially leading to training collapse if not managed.



(a) Grad Norm (with/without `torch.compile`)

(b) Reward (with/without `torch.compile`)

Figure 6: Training collapse potentially caused by `torch.compile` kernels. Disabling it restored stability. Plots show illustrative data.

Base Model The base model for our experiments is QwQ-32B, a 32-billion parameter LLM. We use a context length of up to 32,000 tokens.

Experiments Two main experimental configurations were run:

- Short Experiment: Target thinking lengths for the length reward were sampled from $\{1000, 2000, 3000, 4000\}$ tokens.
- Long Experiment: Target thinking lengths were sampled from $\{2000, 4000, 6000, 8000, 10000\}$ tokens.

Training Details The following training parameters were used:

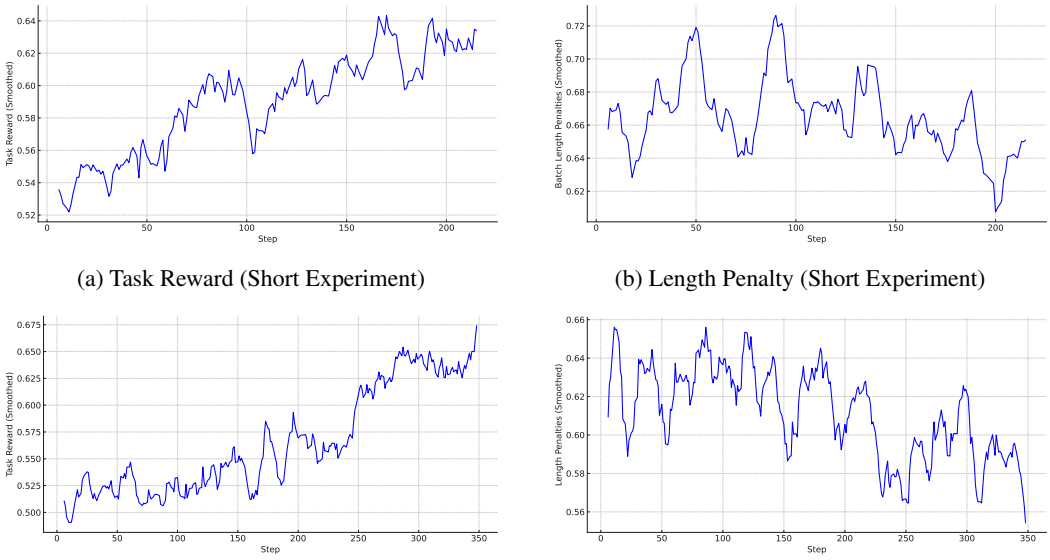
- GRPO parameters: Clipping threshold $\varepsilon = 0.2$, negative advantage ratio cap $\delta = 4$, KL divergence coefficient = 0.001, length reward weight $\alpha = 0.0003$, entropy coefficient = 1e-4.
- Optimizer: AdamW with a learning rate of 3e-7, including a 25-step warmup.
- Batching: 4096 samples per rollout (16 completions × 256 prompts). 8 optimizer steps per batch, with a batch size of 512.
- Asynchrony: Two-step asynchrony was maintained to balance policy freshness and inference–training overlap.
- Efficiency: Sequence packing was used to avoid wasting compute on padding tokens, especially critical for the 32K context length.

5 Results

This section presents the results from the training runs and benchmark evaluations.

5.1 Training Trajectories

Fig. 7 shows the smoothed (10-step moving average) trajectories for task rewards and length penalties for both the Short and Long Experiments.



(a) Task Reward (Short Experiment)

(b) Length Penalty (Short Experiment)

(c) Task Reward (Long Experiment)

(d) Length Penalty (Long Experiment)

Figure 7: Smoothed trajectories (10-step MA) for the runs. (a, c) show a substantial rise in task rewards. (b, d) show a slower decline of length penalties, indicating the model is learning to control response length but may require more training or tuning for precise adherence. (Subfigure references updated for clarity: a is top-left, c is bottom-left)

In both experiments, the task rewards show a substantial and consistent increase over training steps. This indicates that the model is successfully learning to solve the math and coding problems in the training set. The length penalties also show a decline, suggesting the model is learning to adhere to the prompted thinking budget. But, the rate of decline for length penalties is slower than for task rewards, implying that precisely controlling output length is a more challenging aspect for the model to master or might require different weighting for the length penalty.

5.2 Benchmark Performance

We evaluated the model (from the Long Experiment, using prompts like "Think for 10,000 tokens before giving a response") against its baseline QwQ-32B and other relevant models on standard reasoning benchmarks using Evalchemy. The results are presented in Table 1.

Table 1: Benchmark scores (%) on math & coding tasks. INTELLECT-2 (trained from QwQ-32B) shows improvements on AIME and LiveCode (v5) over its baseline.

Model	AIME24	AIME25	LiveCode (v5)	GPQA-Diamond	IFEval
Intellect-2	78.8	64.9	67.8	66.8	81.5
QwQ-32B	76.6	64.8	66.1	66.3	83.4
Qwen-R1-Distill-32B	69.9	58.4	55.1	65.2	72.0
Deepseek-R1	78.6	65.1	64.1	71.6	82.7

The model we trained demonstrates improved performance over its base model QwQ-32B on the AIME mathematical reasoning benchmarks (AIME24: 78.8% vs 76.6%) and the LiveCode (v5)

coding benchmark (67.8% vs 66.1%). Performance on AIME25 is comparable. On GPQA-Diamond and IFEval, QwQ-32B slightly outperforms INTELLECT-2. This could be due to the specific focus of our RL training on math and code tasks, potentially leading to minor degradation on more general instruction following or QA tasks if not explicitly trained for. Overall, the results confirm that our asynchronous RL training approach can effectively enhance the reasoning capabilities of a large pre-trained model on targeted domains.

6 Discussion & Future Work

Our experiments demonstrate the viability and effectiveness of training large reasoning models using asynchronous RL in a decentralized setting. The runs show clear improvements in task-specific reasoning and benchmark performance.

Decentralized RL Scaling A key takeaway is the suitability of this paradigm for tasks with long reasoning chains. As the required length of reasoning (number of "thinking" tokens) increases, the FLOPs required for inference vastly outweigh the FLOPs for training updates. As inference is embarrassingly parallel, decentralization becomes increasingly efficient. This allows for the aggregation of compute from diverse, globally distributed sources, potentially democratizing access to training large-scale reasoning models. The 2-step (or more, as suggested by Fig. 2) asynchrony effectively hides network latency, making global distribution practical.

Challenges and Limitations While promising, this approach has challenges. Managing a large, heterogeneous set of inference workers requires robust infrastructure. Ensuring data quality and security from untrusted nodes is also critical (though not the primary focus of this report, it's an important consideration for production systems). The slower learning of length control compared to task performance suggests that the interplay between different reward components needs careful tuning, or perhaps more sophisticated methods for length control are required. The drop in IFEval scores suggests a potential need for multi-task RL fine-tuning to maintain general capabilities.

Future Work Several avenues for future work emerge:

- Tool-Augmented Reasoning: An exciting direction is to integrate external tools, such as code interpreters, calculators, or external knowledge APIs (e.g., web search), into the reasoning loop. The RL framework can then be used to teach the model how and when to use these tools, potentially leading to much stronger and more reliable reasoning by providing more direct and verifiable reward signals.
- Scaling Decentralization: Further scaling the number of decentralized workers and exploring more advanced strategies for managing staleness and variance in a highly distributed environment.
- Broader Task Domains: Extending this training method to other reasoning-intensive tasks beyond math and code, such as scientific reasoning or complex QA.
- Advanced Reward Shaping: Investigating more nuanced reward functions, potentially incorporating human feedback or model-based reward signals to guide learning more effectively, especially for complex behaviors like precise length control or qualitative aspects of reasoning.

7 Conclusion

This project successfully demonstrated a method for training large-scale reasoning models (up to 32B parameters) using asynchronous reinforcement learning in a decentralized fashion. We showed that policy staleness up to 4 steps has a negligible impact on performance, supporting the feasibility of our approach. Key contributions include data filtering techniques, a novel two-sided GRPO clipping mechanism for improved stability, and empirical validation through the training runs, which improved upon the QwQ-32B baseline in targeted reasoning benchmarks. The findings suggest that decentralized RL, by leveraging the parallelizability of inference for long reasoning chains, offers a scalable and efficient path towards developing more powerful reasoning LLMs.

8 Team Contributions

This project was primarily undertaken by Kushal Thaman. The work was done in collaboration with Justus Mattern and others as part of a research residency at Prime Intellect, who provided conceptual guidance, infrastructure support, and contributed to the broader project from which this report is derived. Specific contributions by me to the work presented in this report include:

- Literature review and formulation of the core research questions related to asynchronous RL for reasoning.
- Design and implementation of specific components of the RL training loop relevant to this report.
- Conducting and analyzing various ablation studies.
- Compiling results and writing this report and the associated poster.

Acknowledgements

I would like to thank the entire team at Prime Intellect for their collaboration, invaluable discussions, and for providing the resources and framework. I also thank the CS224R course staff for their guidance.

References

M. Aggarwal et al. 2025. L1: Controlling Long-Range Reasoning. arXiv:2503.01234. arXiv:2503.01234